

WEEK 8

Task 1

Skill objective:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

Implementation / work:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define HISTORY_CAPACITY 5
#define MAX_CMD_LEN 128

typedef struct {
    char
    entries[HISTORY_CAPACITY][MAX_CMD_LEN];
    int
    count;
    int head;
} HistoryBuffer;

void init_history(HistoryBuffer *hb) {
    hb->count = 0;
    hb->head = 0;
}

void store_command(HistoryBuffer *hb, const char *cmd) {
    if (strlen(cmd) == 0) return;
    int idx = (hb->head + hb->count) % HISTORY_CAPACITY;
    if (hb->count < HISTORY_CAPACITY) {
        strncpy(hb->entries[idx], cmd, MAX_CMD_LEN - 1);
        hb->count++;
    } else {
        /* Circular buffer eviction:
        overwrite oldest entry */
        strncpy(hb->entries[hb->head], cmd, MAX_CMD_LEN - 1);
        hb->head = (hb->head + 1) % HISTORY_CAPACITY;
    }
}

void display_history(const HistoryBuffer *hb) {
    printf("=== Command History Buffer (Capacity %d) ===\n", HISTORY_CAPACITY);
    for (int i = 0; i < hb->count; i++) {
        int idx = (hb->head + i) % HISTORY_CAPACITY;
        printf(" %2d: %s\n", i + 1, hb->entries[idx]);
    }
    printf("Consistency Validation: Stored %d entries accurately.\n", hb->count);
}

int main() {
    HistoryBuffer hb;
    init_history(&hb);

    const char *test_cmds[] = {
        "ls -la /home/teja27",
        "cat /etc/os-release",
        "grep -r 'main' .",
        "gcc -Wall myshell.c -o myshell",
        "./myshell --interactive",
        "ps aux | grep bash",
        "history --validate"
    };

    printf("[Task 1] Storing sequential shell commands into history buffer...\n");
    for (int i = 0; i < 7; i++) {
        store_command(&hb, test_cmds[i]);
    }

    display_history(&hb);
    return 0;
}
Output:
```

```
[Task 1] Storing sequential shell commands into history buffer...
=== Command History Buffer (Capacity 5) ===
 1: grep -r 'main' .
 2: gcc -Wall myshell.c -o myshell
 3: ./myshell --interactive
 4: ps aux | grep bash
 5: history --validate
Consistency Validation: Stored 5 entries accurately.
```

Task 2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

Implementation / work:

```
#include <stdio.h>
```

```

#include <string.h>
#include <stdlib.h>

#define MAX_STAGES 8
#define MAX_ARGS 16

typedef struct {      char
*args[MAX_ARGS];      int
arg_count; }
CommandStage;

typedef struct {      CommandStage
stages[MAX_STAGES];    int
stage_count; } Pipeline;

void parse_pipeline(char *input, Pipeline *pl) {
pl->stage_count = 0;      char *pipe_saveptr;      char
*token = strtok_r(input, "|", &pipe_saveptr);

    while (token != NULL && pl->stage_count < MAX_STAGES) {
CommandStage *st = &pl->stages[pl->stage_count++];      st-
>arg_count = 0;      char *arg_saveptr;      char *arg =
strtok_r(token, " \\t\\n", &arg_saveptr);      while (arg !=
NULL && st->arg_count < MAX_ARGS - 1) {      st-
>args[st->arg_count++] = arg;      arg = strtok_r(NULL,
" \\t\\n", &arg_saveptr);      }      st->args[st-
>arg_count] = NULL;      token = strtok_r(NULL, "|",
&pipe_saveptr);      } }

void print_pipeline_layout(const Pipeline *pl) {      printf("Pipeline Layout
Validation (%d stages):\\n", pl->stage_count);      for (int i = 0; i < pl-
>stage_count; i++) {      printf("[Stage %d: %s]", i + 1, pl-
>stages[i].args[0]);      if (i < pl->stage_count - 1) printf(" ===(pipe
fd)====> ");      }      printf("\\nDetailed Stage Argument Breakdown:\\n");      for
(int i = 0; i < pl->stage_count; i++) {      printf(" Stage %d command: '%s' |
args: ", i + 1, pl->stages[i].args[0]);      for (int j = 0; j < pl-
>stages[i].arg_count; j++) {      printf("[%s] ", pl->stages[i].args[j]);
}      printf("\\n");      } }

int main() {      char command_line[] = "cat sample.log | grep -E 'ERROR|WARN' | cut -d: -f2 | sort
| uniq -c";      printf("Raw Pipeline Input String: \"%s\\n\\n\", command_line);

    Pipeline pl;
    parse_pipeline(command_line, &pl);
    print_pipeline_layout(&pl);      return
0; }

```

Output:

```

Consistency Validation: Stored 3 entries accurately.
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp8ss.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp8ss.c -o exp8ss
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp8ss
Raw Pipeline Input String: "cat sample.log | grep -E 'ERROR|WARN' | cut -d: -f2 | sort | uniq -c"

Pipeline Layout Validation (6 stages):
[Stage 1: cat] ===(pipe fd)====> [Stage 2: grep] ===(pipe fd)====> [Stage 3: WARN'] ===(pipe fd)====> [Stage 4: cut] ===(pipe fd)====> [Stage 5: sort] ===(
pipe fd)====> [Stage 6: uniq]
Detailed Stage Argument Breakdown:
Stage 1 command: 'cat' | args: [cat] [sample.log]
Stage 2 command: 'grep' | args: [grep] [-E] ['ERROR']
Stage 3 command: 'WARN' | args: [WARN']
Stage 4 command: 'cut' | args: [cut] [-d:] [-f2]
Stage 5 command: 'sort' | args: [sort]
Stage 6 command: 'uniq' | args: [uniq] [-c]

```

Skill objective:**Task 3**

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    if (pipe(pipefd) == -1) {
        perror("pipe");
        exit(1);
    }

    printf("[Task 3] Executing connected 2-stage pipeline: 'ls -l' | 'wc -l'\n");

    pid_t pid1 = fork();
    if (pid1 == 0) { /* Child 1:
        Producer ('ls -l') redirects stdout to pipefd[1] */
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        char *args[] = {"ls", "-l", NULL};
        execvp(args[0], args);
        perror("execvp ls");
        exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) { /* Child 2:
        Consumer ('wc -l') redirects stdin from pipefd[0] */
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        char *args[] = {"wc", "-l", NULL};
        execvp(args[0], args);
        perror("execvp wc");
        exit(1);
    }

    /* Parent closes both pipe descriptors and synchronizes */
    close(pipefd[0]);
    close(pipefd[1]);

    int status;
    waitpid(pid1, &status, 0);
    waitpid(pid2, &status, 0);

    printf("[Task 3] Data flow synchronization completed across pipe streams.\n");
    return 0; }
```

Output:

```
naveen123@LAPTOP-4NVQJ2C7:~$ nano exp8sss.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp8sss.c -o exp8sss
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp8sss
[Task 3] Executing connected 2-stage pipeline: 'ls -l' | 'wc -l'
122
[Task 3] Data flow synchronization completed across pipe streams.
charanteja_a@charan:~$
```

Task 4

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

Implementation / work:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    /* 3 commands: 'echo -e ...' | 'grep test' | 'wc -l'
    => 2 pipes */
    int num_cmds = 3;
    int pipes[2][2];
```

Skill objective:

```

for (int i = 0; i < 2; i++) {          if (pipe(pipes[i]) ==
-1) { perror("pipe"); exit(1); }      }

printf("[Task 4] Coordinating multi-stage pipeline: 'echo ...' | 'grep test' | 'wc -l'\n");

/* Child 0: echo */      if (fork() == 0) {          dup2(pipes[0][1], STDOUT_FILENO);
close(pipes[0][0]); close(pipes[0][1]);          close(pipes[1][0]); close(pipes[1][1]);
execlp("echo", "echo", "-e", "test line 1\nother line\ntest line 2\ntest line 3", NULL);
exit(1);      }

/* Child 1: grep */      if (fork() == 0) {
dup2(pipes[0][0], STDIN_FILENO);
dup2(pipes[1][1], STDOUT_FILENO);
close(pipes[0][0]); close(pipes[0][1]);
close(pipes[1][0]); close(pipes[1][1]);
execlp("grep", "grep", "test", NULL);
exit(1);      }

/* Child 2: wc */      if (fork() == 0) {
dup2(pipes[1][0], STDIN_FILENO);
close(pipes[0][0]); close(pipes[0][1]);
close(pipes[1][0]); close(pipes[1][1]);
execlp("wc", "wc", "-l", NULL);
exit(1);      }

/* Parent closes all pipe ends */
for (int i = 0; i < 2; i++) {
close(pipes[i][0]);
close(pipes[i][1]);      }

/* Wait for all child processes */
for (int i = 0; i < num_cmds; i++) {
wait(NULL);      }

printf("[Task 4] Multi-pipe coordination and resource cleanup verified successfully.\n");
return 0; }

```

Output:

```

naveen123@LAPTOP-4NVQJ2C7:~$ nano exp8s4.c
naveen123@LAPTOP-4NVQJ2C7:~$ gcc exp8s4.c -o exp8s4
naveen123@LAPTOP-4NVQJ2C7:~$ ./exp8s4
[Task 4] Coordinating multi-stage pipeline: 'echo ...' | 'grep test' | 'wc -l'
3
[Task 4] Multi-pipe coordination and resource cleanup verified successfully.
naveen123@LAPTOP-4NVQJ2C7:~$ |

```